

```
1 package com.tp.jpjpa;
2
3 import com.tp.jpjpa.model.entities.Categoria;
4 import com.tp.jpjpa.model.entities.Producto;
5 import com.tp.jpjpa.repository.CategoriaRepository;
6 import com.tp.jpjpa.repository.ProductoRepository;
7
8 import java.util.List;
9 import java.util.Optional;
10 import java.util.Scanner;
11 import java.util.logging.Level;
12
13
14 public class Main {
15     public static void main(String[] args) {
16
17
18         //quita los logs de hibernate dejando solo
        los errores.
19         java.util.logging.Logger.getLogger("org.
        hibernate").setLevel(Level.SEVERE);
20
21         Scanner sc = new Scanner(System.in);
22
23         CategoriaRepository categoriaRepo = new
        CategoriaRepository();
24         ProductoRepository productoRepo = new
        ProductoRepository();
25
26         int opcion;
27
28         do {
29             System.out.println("\n");
30             System.out.println("==== MENÚ PRINCIPAL
        ====\n.....");
31             System.out.println("1. Gestión de
        Categorías");
32             System.out.println("2. Gestión de
        Productos");
33             System.out.println("3. Reportes");
34             System.out.println("0. Salir");
```

```
35         System.out.println(  
36         ".....\n");  
37         opcion = intSeguro(sc, "Ingrese un numero  
38         : ");  
39         switch (opcion) {  
40  
41             case 1 -> menuCategorias(sc,  
42             categoriaRepo);  
43             case 2 -> menuProductos(sc,  
44             productoRepo, categoriaRepo);  
45             case 3 -> {  
46  
47                 System.out.println("\n---  
48                 REPORTES ---");  
49                 System.out.println("Elija una  
50                 opcion: ");  
51                 System.out.println("1. Buscar  
52                 Productos por categoria");  
53                 System.out.println("0. Volver al  
54                 menu principal");  
55                 int opcionReporte = intSeguro(sc  
56                 , "Ingrese un numero: \n");  
57  
58                 switch (opcionReporte) {  
59                     case 1 -> {  
60                         mostrarCategoriasActivas(  
61                         categoriaRepo);  
62                         Long idReporte =  
63                         LongSeguro(sc, "\nSelecione ID de categoría: ");  
64                         List<Producto> productos  
65                         = productoRepo.buscarPorCategoria(idReporte);  
66                     }  
67                 }  
68             }  
69         }  
70     }  
71 }
```

```

64         if (productos.stream().
        findAny().isEmpty()) {
65             System.out.println("
        No hay productos activos.");
66         } else {
67             System.out.println("
        Productos de la categoria: ");
68             productos.forEach(p
        ->
69                 System.out.
        println(
70                     "ID
        : " + p.getId() +
71         " | Nombre: " + p.getNombre() +
72         " | Precio: " + p.getPrecio() +
73         " | Descripcion: " + p.getDescripcion() +
74         " | Stock: " + p.getStock() +
75         " | Categoría: " + p.getCategoria().getNombre()
76                     )
77             );}
78
79         }
80         case 0 -> System.out.println
        ("Volviendo al menú principal...");
81
82         }
83
84         }
85
86         case 0 -> System.out.println("
        Saliendo del sistema...");
87
88         default -> System.out.println("
        Opción inválida.");
89     }
90

```

```

91         } while (opcion != 0);
92
93         sc.close();
94     }
95
96     private static void menuCategorias(Scanner sc,
    CategoriaRepository categoriaRepo) {
97
98         int opcion;
99
100        do {
101            System.out.println("\n");
102            System.out.println("\n==== MENÚ DE
    CATEGORÍAS ====");
103            System.out.println("1. Alta de categoría
    ");
104            System.out.println("2. Baja lógica de
    categoría");
105            System.out.println("3. Modificación de
    categoría");
106            System.out.println("4. Listado de
    categorías activas");
107            System.out.println("0. Volver al Menu
    Principal");
108            System.out.println(
    ".....\n");
109
110            opcion = intSeguro(sc, "Ingrese un
    numero: ");
111
112
113            switch (opcion) {
114
115                case 1 -> { //Alta de categoría
116                    System.out.println("\n--- ALTA
    DE CATEGORÍA ---");
117                    System.out.print("Nombre: ");
118                    String nombre = sc.nextLine();
119
120                    if (validarNombreCategoria(
    nombre, categoriaRepo, sc)) {

```

```

121                                     break; // salir del case sin
        pedir descripción
122                                     }
123
124                                     System.out.print("Descripción: "
125                                     );
126                                     String descripcion = sc.nextLine
127                                     ();
128                                     Categoria nueva = Categoria.
129                                     builder()
130                                     .nombre(nombre)
131                                     .descripcion(descripcion
132                                     )
133                                     .build();
134                                     nueva = categoriaRepo.guardar(
135                                     nueva);
136
137                                     System.out.println("Categoría
138                                     creada con ID: " + nueva.getId());
139                                     }
140
141                                     case 2 -> { //Baja logica de
142                                     Categoria
143                                     System.out.println("\n--- BAJA
144                                     LÓGICA DE CATEGORÍA ---");
145
146                                     mostrarCategoriasActivas(
147                                     categoriaRepo);
148
149                                     System.out.print("Elija una
150                                     categoria ");
151
152                                     //ingreso seguro de long
153                                     Long id = LongSeguro(sc, "
154                                     Seleccione ID:");
155
156                                     boolean eliminado =
157                                     categoriaRepo.eliminarLogico(id);

```

```
149
150         if (eliminado) {
151             System.out.println("
Categoría eliminada correctamente.");
152         } else {
153             System.out.println("No
existe una categoría con ese ID.");
154         }
155     }
156
157     case 3 -> { //modificar categoria
158         System.out.println("\n---
MODIFICACIÓN DE CATEGORÍA ---");
159
160         mostrarCategoriasActivas(
            categoriaRepo);
161
162         //revisar
163
164         Long id = LongSeguro(sc, "
Ingrese ID para modificar la categoría: ");
165
166         Optional<Categoria> optionalCat
            = categoriaRepo.buscarPorId(id);
167
168         if (optionalCat.isEmpty() ||
            optionalCat.get().isEliminado()) {
169             System.out.println("No
existe una categoría activa con ese ID.");
170             break;
171         }
172
173         Categoria cat = optionalCat.get
            ();
174
175         System.out.println("Valores
actuales:");
176         System.out.println("Nombre: " +
            cat.getNombre());
177         System.out.println("Descripción
            : " + cat.getDescripcion());
```

```

178
179         System.out.print("Nuevo nombre (
ENTER para mantener): ");
180         String nuevoNombre = sc.nextLine
();
181
182         if (validarNombreCategoria(
nuevoNombre, categoriaRepo, sc)) {
183             break; // salir del case sin
pedir descripción
184         }
185
186         if (!nuevoNombre.isBlank()) cat.
setNombre(nuevoNombre);
187
188         System.out.print("Nueva
descripción (ENTER para mantener): ");
189         String nuevaDesc = sc.nextLine
();
190         if (!nuevaDesc.isBlank()) cat.
setDescripcion(nuevaDesc);
191
192         categoriaRepo.guardar(cat);
193
194         System.out.println("Categoría
modificada correctamente.");
195     }
196
197     case 4 -> {
198         // Listar categorias
199         System.out.println("\n---
LISTADO DE CATEGORÍAS ACTIVAS ---");
200
201         mostrarCategoriasActivas(
categoriaRepo);
202
203     }
204
205
206     case 0 -> System.out.println("
Volviendo al menú principal...");

```

```

207         default -> System.out.println("
    Opción inválida.");
208     }
209
210
211     } while (opcion != 0);
212
213 }
214
215     private static void menuProductos(Scanner sc,
    ProductoRepository productoRepo, CategoriaRepository
    categoriaRepo) {
216
217         int opcion;
218
219         do {
220             System.out.println("\n");
221             System.out.println("\n==== MENÚ DE
    PRODUCTOS ===");
222             System.out.println("1. Alta de producto"
    );
223             System.out.println("2. Baja lógica de
    producto");
224             System.out.println("3. Modificación de
    producto");
225             System.out.println("4. Listado de
    productos activos");
226             System.out.println("0. Volver al Menu
    Principal");
227             System.out.println(
    ".....\n");
228             opcion = intSeguro(sc, "Ingrese un
    numero: ");
229
230             switch (opcion) {
231
232                 case 1 -> { //Alta de producto
233                     System.out.println("\n--- ALTA
    DE PRODUCTO ---");
234
235                     mostrarCategoriasActivas(

```



```

235 categoriaRepo);
236
237             System.out.print("Seleccione ID
de categoría: ");
238             //verificar long
239             Long idCat = LongSeguro(sc, "
Seleccione ID de categoría: ");
240
241             Optional<Categoria> optionalCat
= categoriaRepo.buscarPorId(idCat);
242
243
244             if (optionalCat.isEmpty() ||
optionalCat.get().isEliminado()) {
245                 System.out.println("
Categoría inválida.");
246                 break;
247             }
248
249             Categoria categoria =
optionalCat.get();
250
251             System.out.print("Nombre: ");
252             String nombre = sc.nextLine();
253
254             if (validarNombreProducto(nombre
, productoRepo, sc)) {
255                 break; // salir del case sin
pedir descripción
256             }
257
258 //
259             System.out.print("Descripción: "
);
260             String descripcion = sc.nextLine
();
261
262             System.out.print("Precio: ");
263             double precio = DoubleSeguro(sc
, "Ingresa el precio: ");
264

```

```

265         if (precio <= 0) {
266             System.out.println("El
precio debe ser mayor a 0.");
267             break;
268         }
269
270         System.out.print("Stock: ");
271         int stock = intSeguro(sc, "
Ingrese el stock: ");
272
273         if (stock < 0) {
274             System.out.println("El stock
no puede ser negativo.");
275             break;
276         }
277
278         Producto nuevo = Producto.
builder()
279             .nombre(nombre)
280             .descripcion(descripcion
)
281             .precio(precio)
282             .stock(stock)
283             .disponible(true)
284             .categoria(categoria)
285             .build();
286
287         nuevo = productoRepo.guardar(
nuevo);
288
289         System.out.println("Producto
creado con ID: " + nuevo.getId());
290         System.out.println("Categoría
asignada: " + categoria.getNombre());
291     }
292
293     case 2 -> { //Baja lógica de
producto
294         System.out.println("\n--- BAJA
LÓGICA DE PRODUCTO ---");
295

```

```

296         mostrarProductosActivos(
           productoRepo);
297
298         Long id = LongSeguro(sc, "
Seleccione ID de producto: ");
299
300         Optional<Producto> optionalProd
           = productoRepo.buscarPorId(id);
301
302         if (optionalProd.isEmpty() ||
           optionalProd.get().isEliminado()) {
303             System.out.println("No
existe un producto activo con ese ID.");
304             break;
305         }
306
307         Producto prod = optionalProd.get
           ();
308
309         boolean eliminado = productoRepo
           .eliminarLogico(id);
310
311         if (eliminado) {
312             System.out.println("Producto
dado de baja: " + prod.getNombre());
313         } else {
314             System.out.println("No se
pudo eliminar el producto.");
315         }
316     }
317
318     case 3 -> { //Modificación de
           producto
319         System.out.println("\n---
MODIFICACIÓN DE PRODUCTO ---");
320
321         mostrarProductosActivos(
           productoRepo);
322
323         Long id = LongSeguro(sc, "
Seleccione ID de producto: ");

```

```

324
325         Optional<Producto> optionalProd
    = productoRepo.buscarPorId(id);
326
327         if (optionalProd.isEmpty() ||
    optionalProd.get().isEliminado()) {
328             System.out.println("No
existe un producto activo con ese ID.");
329             break;
330         }
331
332         Producto prod = optionalProd.get
    ();
333
334         System.out.println("Valores
actuales:");
335         System.out.println("Nombre: " +
    prod.getNombre());
336         System.out.println("Precio: " +
    prod.getPrecio());
337         System.out.println("Descripción
    : " + prod.getDescripcion());
338         System.out.println("Stock: " +
    prod.getStock());
339
340         System.out.print("Nuevo nombre (
ENTER para mantener: ");
341         String nuevoNombre = sc.nextLine
    ();
342         if (!nuevoNombre.isBlank()) prod
    .setNombre(nuevoNombre);
343
344         if (validarNombreProducto(
    nuevoNombre, productoRepo, sc)) {
345             break; // salir del case sin
    pedir descripción
346         }
347
348         System.out.print("Nuevo precio (
ENTER para mantener: ");
349         String nuevoPrecio = sc.nextLine

```

```
349 ();
350         if (!nuevoPrecio.isBlank()) {
351             double precio = Double.
parseDouble(nuevoPrecio);
352             if (precio <= 0) {
353                 System.out.println("El
precio debe ser mayor a 0.");
354                 break;
355             }
356             prod.setPrecio(precio);
357         }
358
359         System.out.print("Nueva
descripción (ENTER para mantener): ");
360         String nuevaDescripcion = sc.
nextLine();
361         if (!nuevaDescripcion.isBlank
()) prod.setNombre(nuevaDescripcion);
362
363
364         System.out.print("Nuevo stock (
ENTER para mantener): ");
365         String nuevoStock = sc.nextLine
();
366         if (!nuevoStock.isBlank()) {
367             int stock = Integer.parseInt
(nuevoStock);
368             if (stock < 0) {
369                 System.out.println("El
stock no puede ser negativo.");
370                 break;
371             }
372             prod.setStock(stock);
373         }
374
375         productoRepo.guardar(prod);
376
377         System.out.println("Producto
modificado correctamente.");
378     }
379
```

```
380             case 4 -> { //Listado de productos
                        activos
381                             System.out.println("\n---
LISTADO DE PRODUCTOS ACTIVOS ---");
382
383                             mostrarProductosActivos(
                                productoRepo);
384
385                             }
386
387
388                             // Volver
389                             case 0 -> System.out.println("Volviendo
al menú principal...");
390                             default -> System.out.println("Opción
inválida.");
391                             }
392
393     } while(opcion !=0);
394 }
395
396 private static Double DoubleSeguro(Scanner sc,
String mensaje) {
397     while (true) {
398         System.out.print(mensaje);
399         String input = sc.nextLine();
400
401         try {
402             return Double.parseDouble(input);
403         } catch (NumberFormatException e) {
404             System.out.println("Debe ingresar un
número válido.");
405         }
406     }
407 }
408
409 private static Long LongSeguro(Scanner sc, String
mensaje) {
410     while (true) {
411         System.out.print(mensaje);
412         String input = sc.nextLine();
```

```
413
414         try {
415             return Long.parseLong(input);
416         } catch (NumberFormatException e) {
417             System.out.println("Debe ingresar un
número válido.");
418         }
419     }
420 }
421
422 private static int intSeguro(Scanner sc, String
mensaje) {
423     while (true) {
424         System.out.print(mensaje);
425         String input = sc.nextLine();
426
427         try {
428             return Integer.parseInt(input);
429         } catch (NumberFormatException e) {
430             System.out.println("Debe ingresar un
número válido.");
431         }
432     }
433 }
434
435 private static boolean validarNombreCategoria(String
nombre, CategoriaRepository categoriaRepo, Scanner
sc) {
436     //Validacion de nombre duplicado
437     boolean existe = categoriaRepo.listarActivos()
438         .stream()
439         .anyMatch(c -> c.getNombre().
equalsIgnoreCase(nombre));
440
441     Optional<Categoria> categoriaInactiva =
categoriaRepo.listarInactivos()
442         .stream()
443         .filter(c -> c.getNombre().
equalsIgnoreCase(nombre))
444         .findFirst();
445 }
```

```

446      //verificacion de categoria repetida
447      if (existe) {
448          System.out.println("Ya existe una categoría
con ese nombre.");
449          return true; // no continua
450      }
451      //verificacion de categoria existente pero con
baja logica
452      if (categoriaInactiva.isPresent()) {
453          System.out.println("Ya existe una categoría
con ese nombre, pero se encuentra inactiva" +
454              "\n ¿Desea activarla nuevamente");
455          System.out.println("1. Si");
456          System.out.println("2. No");
457          int opcionAlta = intSeguro(sc, "Seleccione
una opción: ");
458
459          switch (opcionAlta) {
460              case 1 -> {
461                  Categoria cat = categoriaInactiva.
get();
462                  categoriaRepo.AltaLogica(cat.getId
());
463                  System.out.println("Categoría
reactivada correctamente. ID: " + cat.getId());
464                  return true;
465              }
466              case 2 -> {
467                  System.out.println("Operación
cancelada.");
468                  return true; //no continua
469              }
470
471          }
472      }
473      return false; //si pasa las evaluaciones
continua con el alta
474  }
475
476
477 private static void mostrarCategoriasActivas(

```



```
477 CategoriaRepository categoriaRepo) {
478
479     List<Categoria> categorias = categoriaRepo.
        listarActivos();
480
481     if (categorias.stream().findAny().isEmpty()) {
482         System.out.println("No hay categorías
        activas.");
483     } else {
484         System.out.println("Categorías disponibles
        :");
485         categorias.forEach(cat ->
486             System.out.println(
487                 "ID: " + cat.getId() +
488                 " | Nombre: " + cat.
        getNombre() +
489                 " | Descripción: "
        + cat.getDescripcion()
490             );
491     });
492 }
493
494 private static void mostrarProductosActivos(
        ProductoRepository productoRepo) {
495
496     List <Producto> productos = productoRepo.
        listarActivos();
497
498     if (productos.stream().findAny().isEmpty()) {
499         System.out.println("No hay productos activos
        .");
500     } else {
501         productos.forEach(p ->
502             System.out.println(
503                 "ID: " + p.getId() +
504                 " | Nombre: " + p.
        getNombre() +
505                 " | Precio: " + p.
        getPrecio() +
506                 " | Descripción: "
        + p.getDescripcion() +
```

```

507         " | Stock: " + p.
        getStock() +
508         " | Categoría: " + p
        .getCategoria().getNombre()
509     )
510 );
511 }
512 }
513
514 private static boolean validarNombreProducto(String
    nombre, ProductoRepository productoRepo, Scanner sc
    ) {
515     boolean existe = productoRepo.listarActivos()
516         .stream()
517         .anyMatch(c -> c.getNombre().
    equalsIgnoreCase(nombre));
518
519     Optional<Producto> prodInactivo = productoRepo.
    listarInactivos()
520         .stream()
521         .filter(c -> c.getNombre().
    equalsIgnoreCase(nombre))
522         .findFirst();
523
524     //verificacion de producto repetido
525     if (existe) {
526         System.out.println("Ya existe un producto
    con ese nombre.");
527         return true; // no continua
528     }
529
530     //verificacion de producto existente pero con
    baja logica
531     if (prodInactivo.isPresent()) {
532         System.out.println("Ya existe un producto
    con ese nombre, pero se encuentra inactivo" +
533             "\n ¿Desea activarlo nuevamente");//
534         System.out.println("1. Si");
535         System.out.println("2. No");
536         int opcionAltaProd = intSeguro(sc, "
    Seleccione una opción: ");

```

```
537
538         switch (opcionAltaProd) {
539             case 1 -> {
540                 Producto prod = prodInactivo.get();
541                 productoRepo.AltaLogica(prod.getId
542                     ());
543                 System.out.println("Producto
544                 reactivado correctamente. ID: " + prod.getId());
545                 return true; // no continua
546             }
547             case 2 -> {
548                 System.out.println("Operación
549                 cancelada.");
550                 return true; // no continua
551             }
552         }
553         return false; //si pasa las evaluaciones
554         continua con el alta
555     }}
556
557
558
559
560
```

```
1 package com.tp.jp.util;
2
3 import jakarta.persistence.EntityManager;
4 import jakarta.persistence.EntityManagerFactory;
5 import jakarta.persistence.Persistence;
6
7 public class JPAUtil {
8
9     private static final String PERSISTENCE_UNIT = "
    miUnidad";
10
11     private static EntityManagerFactory emf;
12
13     static {
14         try {
15             emf = Persistence.
    createEntityManagerFactory(PERSISTENCE_UNIT);
16         } catch (Exception e) {
17             System.err.println("Error inicializando
    EntityManagerFactory: " + e.getMessage());
18             throw new ExceptionInInitializerError(e);
19         }
20     }
21
22     public static EntityManager getEntityManager() {
23         return emf.createEntityManager();
24     }
25
26     //como usarlo -> EntityManager em = JPAUtil.
    getEntityManager();
27
28     public static EntityManagerFactory
    getEntityManagerFactory() {
29         return emf;
30     }
31
32     public static void close() {
33         if (emf != null && emf.isOpen()) {
34             emf.close();
35         }
36     }
```

37 }

38

```
1 package ar.edu.tup.programacion3.dtos;
2
3 public record UsuarioDTO(
4     String nombre,
5     String apellido,
6     String mail,
7     String celular) {
8     public UsuarioDTO {
9         if (nombre == null) {
10             nombre = "Sin nombre";
11         } else if (apellido == null) {
12             apellido = "Sin apellido";
13         } else if (mail == null) {
14             mail = "Sin mail";
15         } else if (celular == null) {
16             celular = "Sin celular";
17         }
18     }
19 }
20 }
21
22
```

```
1 package ar.edu.tup.programacion3.enums;  
2  
3 public enum Rol {  
4     ADMIN, USUARIO  
5 }  
6
```

```
1 package ar.edu.tup.programacion3.enums;  
2  
3 public enum Estado {  
4     PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO  
5 }  
6
```



```
1 package ar.edu.tup.programacion3.enums;  
2  
3 public enum FormaPago {  
4     TARJETA, TRANSFERENCIA, EFECTIVO  
5 }  
6
```

```
1 package com.tp.jpa.model.entities;
2 import jakarta.persistence.GeneratedValue;
3 import jakarta.persistence.GenerationType;
4 import jakarta.persistence.Id;
5 import jakarta.persistence.MappedSuperclass;
6 import lombok.*;
7 import lombok.experimental.SuperBuilder;
8 import java.time.LocalDateTime;
9
10 @MappedSuperclass
11 @Getter
12 @Setter
13 @ToString
14 @SuperBuilder
15 @AllArgsConstructor
16 @NoArgsConstructor
17 @EqualsAndHashCode(of = {"id"})
18 public abstract class Base {
19
20
21     @Id
22     @GeneratedValue(strategy = GenerationType.
        IDENTITY)
23     private Long id;
24     @Builder.Default
25     protected boolean eliminado = false;
26     @Builder.Default
27     protected LocalDateTime createdAt = LocalDateTime
        .now();
28 }
```

```
1 package com.tp.jpa.model.entities;
2
3 import ar.edu.tup.programacion3.enums.Estado;
4 import ar.edu.tup.programacion3.enums.FormaPago;
5 import jakarta.persistence.*;
6 import lombok.*;
7 import lombok.experimental.SuperBuilder;
8
9 import java.time.LocalDate;
10 import java.util.HashSet;
11 import java.util.Set;
12 @Entity
13 @Table(name = "pedidos")
14 @Getter
15 @Setter
16 @ToString
17 @EqualsAndHashCode(callSuper = true)
18 @AllArgsConstructor
19 @NoArgsConstructor
20 @SuperBuilder
21 public class Pedido extends Base implements
    Calculable {
22     private LocalDate fecha;
23     @Enumerated(EnumType.STRING)
24     private Estado estado;
25     private Double total;
26     @Enumerated(EnumType.STRING)
27     private FormaPago formapago;
28     @ManyToOne
29     @JoinColumn(name = "usuario_id")
30     private Usuario usuario;
31
32     //coleccion de detalle pedidos
33     @Builder.Default
34     @OneToMany(mappedBy = "pedido", cascade =
        CascadeType.ALL, orphanRemoval = true)
35     private Set<DetallePedido> detallePedidos = new
        HashSet<>();
36
37
38     //Agregar detalle
```

```
39     public void addDetallePedido(int cantidad,
    Producto producto) {
40
41         boolean exists = detallePedidos.stream()
42             .anyMatch(detallePedido ->
    detallePedido.getProducto().equals(producto));
43         if (exists) return;
44
45         DetallePedido detallePedido = DetallePedido.
    builder()
46             .cantidad(cantidad)
47             .producto(producto)
48             .subtotal(producto.getPrecio()*
    cantidad)
49             .pedido(this)
50             .build();
51         detallePedidos.add(detallePedido);
52
53     }
54
55     //Buscar detalle por producto
56     public DetallePedido findDetallePedidoByProducto(
    Producto producto) {
57         return detallePedidos.stream()
58             .filter(detallePedido ->
    detallePedido.getProducto().equals(producto))
59             .findFirst()
60             .orElse(null);
61     }
62
63
64     //Eliminar detalle por producto
65     public void deleteDetalleByProducto(Producto
    producto) {
66         detallePedidos.removeIf(d -> d.getProducto().
    equals(producto));
67     }
68
69
70     @Override
71     public void calcularTotal() {
```

```
72         double totalFinal = detallePedidos.stream()
73             .mapToDouble(DetallePedido::
        getSubtotal)
74             .sum();
75         this.total = totalFinal;
76         System.out.println("Total del pedido: " +
        totalFinal);
77     }
78
79
80
81 }
```

```
1 package com.tp.jpamodel.entities;
2 import ar.edu.tup.programacion3.enums.Rol;
3 import jakarta.persistence.*;
4 import lombok.*;
5 import lombok.experimental.SuperBuilder;
6
7 import java.util.*;
8
9 @Entity
10 @Table(name = "usuarios")
11 @Getter
12 @Setter
13
14 @ToString(callSuper = true, exclude = "contrasenia")
15 @EqualsAndHashCode(callSuper = false, of = {"mail"})
16
17 @SuperBuilder
18 @NoArgsConstructor
19 @AllArgsConstructor
20 public class Usuario extends Base {
21     private String nombre;
22     private String apellido;
23     @Column(unique = true)
24     private String mail;
25     @Column(unique = true)
26     private String celular;
27     private String contrasenia;
28     @Enumerated(EnumType.STRING)
29     private Rol rol;
30
31     //sets de pedidos
32
33
34     @Builder.Default
35     @OneToMany(mappedBy = "usuario")
36     private Set<Pedido> pedidos = new HashSet<>();
37     public void addPedido(Pedido pedido) {
38         pedidos.add(pedido);
39     }
40
41
```

42

43 }

```
1 package com.tp.jpamodel.entities;
2
3 import jakarta.persistence.*;
4 import lombok.*;
5 import lombok.experimental.SuperBuilder;
6
7 @Entity
8 @Table(name = "productos")
9 @Getter
10 @Setter
11 @ToString
12 @EqualsAndHashCode(callSuper = false , of = {"nombre"
13     , "precio"})
13 @SuperBuilder
14 @NoArgsConstructor
15 @AllArgsConstructor
16 public class Producto extends Base {
17     @Column(unique = true)
18     private String nombre;
19     private Double precio;
20     private String descripcion;
21     private int stock;
22     private String imagen;
23     private boolean disponible;
24     @ManyToOne
25     @JoinColumn(name = "categoria_id")
26     private Categoria categoria;
27
28
29 }
```



```
1 package com.tp.jpa.model.entities;
2
3 import jakarta.persistence.*;
4 import lombok.*;
5 import lombok.experimental.SuperBuilder;
6
7 import java.util.HashSet;
8 import java.util.Set;
9
10 @Entity
11 @Table(name = "categorias")
12 @SuperBuilder
13 @Getter
14 @Setter
15 @ToString(callSuper = true, exclude = "productos")
16 @EqualsAndHashCode(callSuper = false, of = {"nombre"
    })
17 @AllArgsConstructor
18 @NoArgsConstructor
19 public class Categoria extends Base {
20     @Column(unique = true)
21     private String nombre;
22     private String descripcion;
23     @OneToMany(mappedBy = "categoria", cascade =
        CascadeType.ALL, orphanRemoval = true)
24     @Builder.Default
25     private Set<Producto> productos = new HashSet
        <>();
26
27     //helper para mantener la bidireccionalidad
28     public void addProducto(Producto producto) {
29         productos.add(producto);
30         producto.setCategoria(this);
31     }
32 }
33
34
35
36
```

```
1 package com.tp.jpa.model.entities;
2
3 public interface Calculable{
4     void calcularTotal();
5 }
6
7
```

```
1 package com.tp.jpa.model.entities;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.JoinColumn;
5 import jakarta.persistence.ManyToOne;
6 import jakarta.persistence.Table;
7 import lombok.*;
8 import lombok.experimental.SuperBuilder;
9
10 @Entity
11 @Table(name = "detalle_pedidos")
12 @Getter
13 @Setter
14 @EqualsAndHashCode(callSuper = false, of = {"producto
    "})
15 @NoArgsConstructor
16 @AllArgsConstructor
17 @SuperBuilder
18 public class DetallePedido extends Base {
19
20     private int cantidad;
21     private Double subtotal;
22     @ManyToOne
23     @JoinColumn(name = "producto_id")
24     private Producto producto;
25     @ManyToOne
26     @JoinColumn(name = "pedido_id", nullable = false)
27     private Pedido pedido;
28
29     public DetallePedido(int cantidad, Double
    subtotal) {
30         this.cantidad = cantidad;
31         this.subtotal = subtotal;
32
33     }
34 }
```

```
1 package com.tp.jpa.repository;
2 import com.tp.jpa.model.entities.Base;
3 import com.tp.jpa.util.JPAUtil;
4 import jakarta.persistence.EntityManagerFactory;
5
6
7 import java.util.List;
8 import java.util.Optional;
9
10 public abstract class BaseRepository<T extends Base
    > {
11
12     protected final Class<T> entityClass;
13     protected final EntityManagerFactory emf;
14
15     public BaseRepository(Class<T> entityClass) {
16         this.entityClass = entityClass;
17         this.emf = JPAUtil.getEntityManagerFactory();
18     }
19
20     // ===== 1. Guardar (T entity) =====
21     public T guardar(T entity) {
22         var em = emf.createEntityManager();
23         var tx = em.getTransaction();
24
25         try {
26             tx.begin();
27             T merged = em.merge(entity);
28             tx.commit();
29             return merged;
30         } catch (Exception e) {
31             if (tx.isActive()) {
32                 tx.rollback();
33             }
34             throw new RuntimeException("Error al
guardar la entidad en BaseRepository", e);
35         } finally {
36             em.close();
37         }
38     }
39
```

```
40 // ===== 2. buscarPorId(Long id) =====
41 public Optional<T> buscarPorId(Long id) {
42     var em = emf.createEntityManager();
43
44     try {
45         T entity = em.find(entityClass, id);
46         return Optional.ofNullable(entity); //
        retorna empty si es null
47
48     } finally {
49         em.close();
50     }
51 }
52
53 // ===== 3. listarActivos =====
54 public List<T> listarActivos() {
55     var em = emf.createEntityManager();
56
57     try {
58         String jpql = "SELECT e FROM " +
        entityClass.getSimpleName() + " e WHERE e.eliminado
        = false";
59         return em.createQuery(jpql, entityClass).
        getResultList();
60
61     } finally {
62         em.close();
63     }
64 }
65
66 // ===== 3. listarInactivos =====
67 public List<T> listarInactivos() {
68     var em = emf.createEntityManager();
69
70     try {
71         String jpql = "SELECT e FROM " +
        entityClass.getSimpleName() + " e WHERE e.eliminado
        = true";
72         return em.createQuery(jpql, entityClass).
        getResultList();
73 }
```

```
74         } finally {
75             em.close();
76         }
77     }
78
79
80     // ===== 4. eliminarLogico =====
81
82     public boolean eliminarLogico(Long id) {
83         var em = emf.createEntityManager();
84         var tx = em.getTransaction();
85
86         try {
87             T entity = em.find(entityClass, id);
88
89             if (entity == null) {
90                 return false; //mo existe -> retorna
falso
91             }
92
93             tx.begin();
94             entity.setEliminado(true);
95             em.merge(entity);
96             tx.commit();
97
98             return true;
99
100         } catch (Exception e) {
101             if (tx.isActive()) {
102                 tx.rollback();
103             }
104             throw new RuntimeException("Error al
eliminar lógicamente la entidad", e);
105
106         } finally {
107             em.close();
108         }
109     }
110
111     // Alta logixa
112
```

```
113     public boolean AltaLogica(Long id) {
114         var em = emf.createEntityManager();
115         var tx = em.getTransaction();
116
117         try {
118             T entity = em.find(entityClass, id);
119
120             if (entity == null) {
121                 return false; //no existe -> retorna
122                 falso
123             }
124
125             tx.begin();
126             entity.setEliminado(false);
127             em.merge(entity);
128             tx.commit();
129
130             return true;
131         } catch (Exception e) {
132             if (tx.isActive()) {
133                 tx.rollback();
134             }
135             throw new RuntimeException("Error al dar
136 de alta la entidad la entidad", e);
137         } finally {
138             em.close();
139         }
140     }
141
142
143 }
144
```

```
1 package com.tp.jpa.repository;
2
3 import com.tp.jpa.model.entities.Producto;
4 import jakarta.persistence.EntityManager;
5
6 import java.util.List;
7
8 public class ProductoRepository extends
  BaseRepository<Producto> {
9
10     public ProductoRepository() {
11         super(Producto.class);
12     }
13
14     public List<Producto> buscarPorCategoria(Long
  categoriaId) {
15         var em = emf.createEntityManager();
16
17         try {
18             String jpql = "SELECT p FROM Producto p "
19 +
20             "WHERE p.categoria.id = :
  categoriaId " +
21             "AND p.eliminado = false";
22
23             return em.createQuery(jpql, Producto.
  class)
24                 .setParameter("categoriaId",
  categoriaId)
25                 .getResultList();
26         } finally {
27             em.close();
28         }
29     }
30
31     public List<Producto> buscarPorNombre(String
  nombre) {
32         EntityManager em = emf.createEntityManager();
33         try {
34             return em.createQuery(
```



```
35         "SELECT p FROM Producto p
        WHERE p.nombre = LOWER(:nombre)",
36         Producto.class
37     ).setParameter("nombre", nombre)
38     .getResultList();
39     } finally {
40         em.close();
41     }
42 }
43
44 }
45
```

```
1 package com.tp.jp.a.repository;  
2  
3 import com.tp.jp.a.model.entities.Base;  
4 import com.tp.jp.a.model.entities.Categoria;  
5  
6  
7  
8 public class CategoriaRepository extends  
   BaseRepository<Categoria> {  
9  
10     public CategoriaRepository() {  
11         super(Categoria.class);  
12     }  
13 }
```

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <persistence xmlns="https://jakarta.ee/xml/ns/
  persistence"
3         xmlns:xsi="http://www.w3.org/2001/
  XMLSchema-instance"
4         xsi:schemaLocation="https://jakarta.ee/
  xml/ns/persistence
5 https://jakarta.ee/xml/ns/persistence/persistence_3_0
  .xsd" version="3.0">
6     <persistence-unit name="miUnidad"
7         transaction-type="
  RESOURCE_LOCAL">
8         <provider>org.hibernate.jpa.
  HibernatePersistenceProvider</provider>
9         <!-- CLASES -->
10        <class>com.tp.jpa.model.entities.Usuario</
  class>
11        <class>com.tp.jpa.model.entities.Pedido</
  class>
12        <class>com.tp.jpa.model.entities.
  DetallePedido</class>
13        <class>com.tp.jpa.model.entities.Producto</
  class>
14        <class>com.tp.jpa.model.entities.Categoria</
  class>
15
16        <properties>
17            <!-- JDBC H2 en archivo -->
18            <property name="jakarta.persistence.jdbc.
  driver"
19                value="org.h2.Driver"/>
20            <property name="jakarta.persistence.jdbc.
  url"
21                value="jdbc:h2:file:./data/
  jpa_db"/>
22            <property name="jakarta.persistence.jdbc.
  user" value="sa"/>
23            <property name="jakarta.persistence.jdbc.
  password" value=""/>
24            <!-- Dialecto -->
25            <property name="hibernate.dialect"

```

```
26         value="org.hibernate.dialect.  
H2Dialect"/>  
27         <!-- Hibernate -->  
28         <property name="hibernate.hbm2ddl.auto"  
value="update"/>  
29         <!-- Se cambia a false para no mostrar  
los SQL de las busquedas -->  
30         <property name="hibernate.show_sql" value  
="false"/>  
31         <property name="hibernate.format_sql"  
value="false"/>  
32         <property name="hibernate.  
use_sql_comments" value="false"/>  
33     </properties>  
34 </persistence-unit>  
35 </persistence>
```